ĐẶC TẢ THIẾT KẾ gemm_local_sram

Mục lục

1 Tổng quan **2**
1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2
1.4 Điều kiện phục vụ đồng thời . . . 5

2 Đặc tả thanh ghi **5**

3 Mô tả chức năng **5**
3.1 Bố cục logic . . . 5
3.2 Tổ chức vật lý và ánh xạ . . . 5
3.3 Module con . . . 6
3.4 Timing . . . 7
3.5 Reset, hiện thực, kiểm chứng . . . 8

1. Tổng quan

1.1. Tính năng chính

Scratchpad giữ bốn buffer logic A, B, C, BIAS, mỗi buffer có NB bank. Phía tính (array, VPU) cần đọc hoặc ghi P phần tử trong một chu kỳ, nên mỗi bank được chia thành nhiều **sub-bank** một cổng sao cho mỗi truy cập song song chạm các sub-bank khác nhau. Phía DMA đọc hoặc ghi một word mỗi chu kỳ. Hai phía dùng hai bank khác nhau (ping-pong) hoặc hai pha khác nhau (không ping-pong), nên không bao giờ tranh nhau một sub-bank.

Chức năng

- Mỗi bank có dung lượng: A 256 word, B 256 word, C 1024 word, BIAS 32 word; tổng NB $\times$ 1568 word (NB = 2: 12,25 KiB)
- Mọi cổng đọc có độ trễ cố định 1 chu kỳ; mọi lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ có lệnh; không có backpressure
- Mỗi sub-bank vật lý là RAM một cổng đồng bộ. Bank b của buffer X chỉ bị truy cập bởi một phía trong một chu kỳ ; hai bank khác nhau được truy cập độc lập cùng chu kỳ
- Nội dung không được khởi tạo bởi `rst_n` hay `soft_clr`

1.2. Tham số

Bảng 1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	4 hoặc 8
`HAS_PINGPONG`	1	1: NB = 2; 0: NB = 1
`NB, Q, LP`	2, 2, 3	localparam: số bank; Q = P/4 (số sub-bank của B); LP = $\log_2 P$

1.3. Tín hiệu giao tiếp

Bảng 2: Tín hiệu giao tiếp của `gemm_local_sram`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Cổng ghi phía DMA (LOAD_A, LOAD_B, LOAD_BIAS)			
`i_dw_valid`	1	Input	Yêu cầu ghi một word 32 bit vào scratchpad. Ghi được nhận khi valid và ready cùng bằng 1 tại cạnh lên.
`i_dw_buf`	2	Input	Chọn buffer đích cho word ghi. Giá trị 0: A. Giá trị 1: B. Giá trị 2: BIAS; giá trị 3 không hợp lệ.

Bảng 2: Tín hiệu giao tiếp của `gemm_local_sram` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_dw_bank`	1	Input	Chọn bank đích cho word ghi. Hợp lệ cùng yêu cầu ghi; bỏ qua khi chỉ có một bank.
`i_dw_addr`	10	Input	Địa chỉ word logic trong buffer đích. A/B dùng miền 0 đến 255; BIAS dùng miền 0 đến 31. Hợp lệ cùng yêu cầu ghi.
`i_dw_data`	32	Input	Word dữ liệu 32 bit cần ghi vào scratchpad. Hợp lệ cùng yêu cầu ghi; DMA giữ nguyên thứ tự byte của dữ liệu DRAM.
`o_dw_ready`	1	Output	Báo scratchpad có thể nhận word ghi. Thiết kế SRAM hiện tại luôn giữ bằng 1; giao dịch vẫn được xác định bởi valid và ready.
Cổng đọc phía DMA (STORE_C)			
`i_dr_valid`	1	Input	Yêu cầu đọc một word từ buffer C. Yêu cầu được nhận tại cạnh lên khi valid và ready cùng bằng 1.
`i_dr_bank`	1	Input	Chọn bank nguồn của buffer C. Hợp lệ cùng yêu cầu đọc; bỏ qua khi chỉ có một bank.
`i_dr_addr`	10	Input	Địa chỉ word logic cần đọc trong buffer C. Hợp lệ cùng yêu cầu đọc; miền địa chỉ phụ thuộc bố cục INT32 hoặc INT8 của C.
`o_dr_ready`	1	Output	Báo scratchpad có thể nhận yêu cầu đọc C. Thiết kế SRAM hiện tại luôn giữ bằng 1.
`o_dr_rvalid`	1	Output	Báo word đọc trả về hợp lệ. Xuất hiện đúng một chu kỳ sau yêu cầu được nhận; mỗi yêu cầu có đúng một phản hồi, theo thứ tự.
`o_dr_rdata`	32	Output	Word dữ liệu 32 bit đọc từ buffer C. Bên nhận lấy dữ liệu khi read rvalid = 1; không có ready cho kênh trả dữ liệu.
Bank phía tính (từ FSM)			
`i_comp_bank_a`	1	Input	Chọn bank của buffer A dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`i_comp_bank_b`	1	Input	Chọn bank của buffer B dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`i_comp_bank_bias`	1	Input	Chọn bank của buffer BIAS dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`i_comp_bank_c`	1	Input	Chọn bank của buffer C dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`i_comp_active`	1	Input	Báo COMPUTE đang hoạt động. SRAM dùng tín hiệu này để kiểm tra quyền truy cập bằng assertion; tín hiệu không thay thế các enable đọc/ghi.
Cổng đọc A, B (từ array)			
`i_a_rd_en`	1	Input	Cho phép đọc một vector A. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.

Bảng 2: Tín hiệu giao tiếp của `gemm_local_sram` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_a_rd_row`	5	Input	Hàng đầu i0 của vector A. Là bội của P; hợp lệ cùng read enable.
`i_a_rd_k`	5	Input	Chỉ số k của cột A cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`o_a_rd_data`	8P	Output	Vector P byte INT8 của A. Lane r tại bit [8r +: 8] chứa A[i0+r][k]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ có thể chứa giá trị bất kỳ.
`i_b_rd_en`	1	Input	Cho phép đọc một vector B. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.
`i_b_rd_k`	5	Input	Chỉ số k của hàng B cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`i_b_rd_col`	5	Input	Cột đầu j0 của vector B. Là bội của P; hợp lệ cùng read enable.
`o_b_rd_data`	8P	Output	Vector P byte INT8 của B. Lane c tại bit [8c +: 8] chứa B[k][j0+c]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ có thể chứa giá trị bất kỳ.
Cổng BIAS và C (từ VPU)			
`i_bias_rd_en`	1	Input	Yêu cầu đọc P phần tử bias. Bằng 1 khi đoạn hàng hợp lệ và hậu xử lý được bật; dữ liệu về sau một chu kỳ.
`i_bias_rd_col`	5	Input	Cột đầu j0 của vector bias. Là bội của P; hợp lệ cùng bias read enable.
`o_bias_rd_data`	32P	Output	Vector P giá trị bias INT32. Lane c tại bit [32c +: 32] chứa bias[j0+c]. Hợp lệ đúng một chu kỳ sau yêu cầu; lane bị mask có thể chứa giá trị bất kỳ.
`i_c_wr_en`	1	Input	Cho phép ghi một đoạn hàng kết quả vào buffer C tại cạnh lên. Các trường fmt, row, col, data và mask phải hợp lệ trong chu kỳ này.
`i_c_wr_fmt`	1	Input	Chọn định dạng phần tử ghi C. Giá trị 0: INT32, mỗi phần tử là một word. Giá trị 1: INT8, mỗi phần tử là một byte. Hợp lệ cùng write enable.
`i_c_wr_row`	5	Input	Chỉ số hàng i của buffer C cần ghi. Hợp lệ cùng write enable.
`i_c_wr_col`	5	Input	Cột đầu j0 của đoạn hàng cần ghi. Là bội của P; hợp lệ cùng write enable.
`i_c_wr_data`	32P	Input	Dữ liệu ghi đoạn hàng C. Với fmt = 0, phần tử c nằm tại bit [32c +: 32]. Với fmt = 1, phần tử c nằm tại bit [8c +: 8] và phần cao còn lại bằng 0. Hợp lệ cùng write enable.
`i_c_wr_mask`	P	Input	Cho phép ghi theo phần tử. Bit c = 1 ghi phần tử c theo định dạng fmt; bit c = 0 giữ nguyên phần tử đó. Hợp lệ cùng write enable.

Giao tiếp

- `o_dw_ready` và `o_dr_ready` luôn bằng 1. Lệnh chỉ được nhận khi valid và ready cùng bằng 1. Mỗi chu kỳ nhận một word ghi và một yêu cầu đọc; mỗi yêu cầu đọc có đúng một phản hồi theo thứ tự.
- `o_a_rd_data`, `o_b_rd_data`, `o_bias_rd_data` hợp lệ ở đúng chu kỳ sau chu kỳ có `*_rd_en`; ở chu kỳ khác giá trị tùy ý. `o_dr_rvalid` là thanh ghi
- Lane ứng với hàng hoặc cột ngoài vùng hợp lệ của tác vụ mang nội dung tùy ý

1.4. Điều kiện phục vụ đồng thời

Trong một chu kỳ SRAM phải phục vụ đồng thời một lần đọc A (P byte từ P hàng), một lần đọc B (P byte liền nhau), một lần đọc BIAS (P word) và một lần ghi C (P phần tử có mask). Hai phía tính và DMA được chạy đồng thời trên các bank khác nhau. Khi NB = 1, môi trường chỉ cho phép một phía hoạt động tại một thời điểm. Đọc A/B/BIAS trả dữ liệu ở chu kỳ kế tiếp; bên nhận chỉ dùng các lane hợp lệ. Ghi C với fmt = 0 lấy phần tử c từ data[32c +: 32] và mask[c] cho phép ghi cả word. Với fmt = 1, lấy byte c từ data[8c +: 8], mask[c] cho phép ghi byte đó; phần cao của data bằng 0. Hàng/cột đầu phải hợp lệ và cột đầu là bội của P. Mask chặn các phần tử ngoài miền tác vụ. Mỗi yêu cầu đọc DMA được nhận phải tạo đúng một rvalid ở chu kỳ tiếp theo, theo đúng thứ tự; không có ready cho dữ liệu trả về. Nội dung RAM và các lane không hợp lệ không cần khởi tạo, và được phép chứa X trong mô phỏng.

2. Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Cấu hình được nhận qua các tín hiệu đầu vào; trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3. Mô tả chức năng

3.1. Bố cục logic

Bảng 3: Địa chỉ word logic trong mỗi bank

Buffer	Địa chỉ a	Nội dung word
A	{i[4:0], kw[2:0]}	Byte e = $A[i][4kw + e]$
B	{k[4:0], jw[2:0]}	Byte e = $B[k][4jw + e]$
BIAS	j[4:0]	$bias[j]$
C, INT32	{i[4:0], j[4:0]}	$C[i][j]$
C, INT8	{i[4:0], jw[2:0]}	Byte e = $Y[i][4jw + e]$ (dùng word 0 … 255)

3.2. Tổ chức vật lý và ánh xạ

Bảng 4: Sub-bank của mỗi bank

Buffer	Số sub-bank	Sâu × rộng	Sub-bank của a	Địa chỉ cục bộ
A	P	(256/P) × 32	i mod P	{i[4:LP], kw}
B	Q = P/4	(256/Q) × 32	jw mod Q	{k, jw » $\log_2$Q}
BIAS	P	(32/P) × 32	j mod P	j » LP
C	P	(1024/P) × 32, ghi theo byte	a mod P	a » LP

Bảng 5: Số RAM vật lý

Cấu hình	A	B	BIAS	C
P = 4, NB = 1	4 × 64	1 × 256	4 × 8	4 × 256

Bảng 5: Số RAM vật lý (tiếp theo)

Cấu hình	A	B	BIAS	C
P = 8, NB = 1	8 × 32	2 × 128	8 × 4	8 × 128
P = 8, NB = 2	16 × 32	4 × 128	16 × 4	16 × 128

3.2.1. Truy cập phía tính

Cổng phía tính (b = `i_comp_bank_*`)

- Đọc A (i_0, k): sub-bank r (0 ≤ r < P) của A[b] đọc địa chỉ {i_0[4:LP], k[4:2]}. Lane r = byte k[1:0] của sub-bank r; k[1:0] được chốt để chọn byte ở chu kỳ sau
- Đọc B (k, j_0): sub-bank s (0 ≤ s < Q) của B[b] đọc {k, (j_0 » 2) » $\log_2$Q}. Lane c = byte (c mod 4) của sub-bank c/4
- Đọc BIAS (j_0): sub-bank c của BIAS[b] đọc j_0 » LP; lane c = sub-bank c
- Ghi C INT32 (i, j_0): lane c ghi sub-bank c của C[b] tại ({i, j_0} » LP), cả 4 byte, nếu `mask[c]`
- Ghi C INT8 (i, j_0): word thứ w (0 ≤ w < Q) của đoạn hàng có địa chỉ a = {i, j_0/4 + w}, nằm ở sub-bank s = (j_0/4 + w) mod P, địa chỉ a » LP. Sub-bank s nhận word w = (s − j_0/4) mod P nếu w < Q; byte e của nó = `i_c_wr_data[8(4w+e)+:8]`, byte enable e = `mask[4w+e]`

3.2.2. Truy cập phía DMA

Cổng DMA

- Ghi `i_dw_*`: sub-bank và địa chỉ cục bộ theo bảng Sub-bank của mỗi bank với a = `i_dw_addr`, trong buffer `i_dw_buf`, bank `i_dw_bank`; cả 4 byte
- Đọc `i_dr_*`: sub-bank (a mod P) của C[`i_dr_bank`], địa chỉ a » LP. Số sub-bank và bank được chốt để chọn dữ liệu ra ở chu kỳ sau

> **VÌ SAO KHÔNG BAO GIỜ XUNG ĐỘT**
> i_0 và j_0 là bội của P, nên P hàng của một đọc A rơi vào P sub-bank khác nhau, P cột của một đọc BIAS hoặc ghi C INT32 cũng vậy; Q word của một đọc B hoặc ghi C INT8 nằm liền nhau nên rơi vào Q sub-bank khác nhau. Ở INT8, {i, jw} = 8i + jw và 8i chia hết cho P, nên sub-bank chỉ phụ thuộc jw. Mô hình kiểm đủ mọi (i_0, k), (k, j_0), mọi đoạn hàng C, với P = 4, 8.

3.2.3. Chọn bên truy cập

Mux cổng của mỗi sub-bank (X, b, s)

- `dma_sel` = lệnh DMA (ghi `i_dw_*` cho A, B, BIAS; đọc `i_dr_*` cho C) có buffer X, bank b và sub-bank s
- `cmp_sel` = lệnh phía tính tương ứng (`i_a_rd_en`, `i_b_rd_en`, `i_bias_rd_en`, `i_c_wr_en` với byte enable khác 0 cho s) và `i_comp_bank_X` = b
- RAM: `en=dma_selvcmp_sel`; địa chỉ, dữ liệu, byte enable lấy từ bên được chọn. Nếu cả hai , DMA thắng
- Không cần tín hiệu pha: quyền truy cập do bank và lịch của FSM bảo đảm

3.3. Module con

Bảng 6: Tín hiệu giao tiếp của `gemm_spm_ram` (tham số DEPTH, BE)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`i_en`	1	Input	Cho phép truy cập RAM tại cạnh lên. Khi bằng 0, không có lệnh đọc hoặc ghi mới.

Bảng 6: Tín hiệu giao tiếp của `gemm_spm_ram` (tham số DEPTH, BE) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_we`	1	Input	Chọn thao tác RAM khi i_en = 1. Giá trị 1: ghi; 0: đọc đồng bộ.
`i_be`	4	Input	Cho phép ghi từng byte của word. Khi tham số BE = 1, bit b điều khiển byte b; khi BE = 0, bỏ qua mặt nạ và ghi cả word.
`i_addr`	$\lceil \log_2 \text{DEPTH} \rceil$	Input	Địa chỉ word trong RAM con, từ 0 đến DEPTH − 1. Được lấy mẫu cùng i_en tại cạnh lên.
`i_wdata`	32	Input	Word 32 bit ghi vào RAM khi i_en và i_we cùng bằng 1. Các byte thực sự được cập nhật theo mặt nạ i_be.
`o_rdata`	32	Output	Word 32 bit đọc từ RAM. Hợp lệ một chu kỳ sau lệnh có i_en = 1 và i_we = 0.

Bảng 7: Module con theo buffer

Module	Nội dung
`gemm_spm_a`	NB × P RAM; phục vụ đọc vector A và ghi word DMA; chốt k[1:0] và bank để chọn byte trả về.
`gemm_spm_b`	NB × Q RAM; phục vụ đọc vector B và ghi word DMA; chốt bank để chọn dữ liệu trả về.
`gemm_spm_bias`	NB × P RAM; dùng thanh ghi vì mỗi RAM chỉ sâu 4 hoặc 8 word; đọc vector bias và ghi word DMA.
`gemm_spm_c`	NB × P RAM có byte enable; ghi đoạn hàng INT32/INT8 và đọc word DMA; xoay word khi ghi INT8, chốt bank và sub-bank để chọn dữ liệu đọc.

```
// gemm_spm_ram: RAM một cổng, đọc đồng bộ (tham khảo)
logic [31:0] mem [DEPTH];
always_ff @(posedge clk)
  if (i_en) begin
    if (i_we) begin
      for (int e = 0; e < 4; e++)
        if (!BE || i_be[e]) mem[i_addr][8*e +: 8] <= i_wdata[8*e +: 8];
    end else
      o_rdata <= mem[i_addr];
  end
```

3.4. Timing

Bảng 8: Thời điểm (t = chu kỳ có lệnh)

Cổng	Lệnh	Kết quả
A, B, BIAS đọc	t	dữ liệu ở t + 1
C ghi (VPU), A/B/BIAS ghi (DMA)	t	có trong RAM từ t + 1
C đọc (DMA)	t	`o_dr_rvalid` = 1, dữ liệu ở t + 1

3.5. Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0 hoặc `i_soft_clr`: `o_dr_rvalid` về 0. Thanh ghi chọn byte, bank, sub-bank và nội dung RAM không bị reset

Hiện thực

- RAM viết theo mẫu `gemm_spm_ram` để công cụ suy ra block RAM hoặc LUTRAM có đọc đồng bộ. Không được dùng RAM đọc bất đồng bộ cho A, B, C
- Không có mux tổ hợp từ `*_rd_en` tới dữ liệu ra; đường ra là: thanh ghi đầu ra RAM $\rightarrow$ mux bank $\rightarrow$ mux byte/lane
- Mỗi buffer trong file riêng; NB và số sub-bank dựng bằng `generate`

Assertion bắt buộc (SVA)

- `i_dw_valid` $\rightarrow$ `i_dw_buf` $\neq 3 \wedge$ địa chỉ trong miền của buffer (A, B < 256; BIAS < 32)
- NB = 2: `i_dw_valid` $\wedge$ `i_comp_active` $\rightarrow$ `i_dw_bank` $\neq$ `i_comp_bank_X` với X = `i_dw_buf`; và `i_dr_valid` $\wedge$ `i_comp_active` $\rightarrow$ `i_dr_bank` $\neq$ `i_comp_bank_c`. NB = 1: `i_dw_valid` $\vee$ `i_dr_valid` $\rightarrow \neg$`i_comp_active`
- `i_a_rd_en` $\vee$ `i_b_rd_en` $\vee$ `i_bias_rd_en` $\vee$ `i_c_wr_en` $\rightarrow$ `i_comp_active`
- Không sub-bank nào có `dma_sel` $\wedge$ `cmp_sel` cùng chu kỳ
- `i_a_rd_row`, `i_b_rd_col`, `i_bias_rd_col`, `i_c_wr_col` là bội của P khi có lệnh

> **KIỂM CHỨNG BẰNG MÔ HÌNH**
> `gemm_sram_model.py`: ánh xạ sub-bank và địa chỉ cục bộ là song ánh với P = 4, 8; 240 cấu hình ngẫu nhiên (M, N, K, bank, NB = 1, 2) nạp A, B, BIAS qua cổng DMA rồi đọc qua cổng array/VPU, ghi C INT32 và INT8 qua cổng VPU rồi đọc qua cổng DMA: mọi giá trị khớp và không truy cập song song nào chạm hai lần một sub-bank.